RELAZIONE ESERCITAZIONE 1 – Alessio Attilia

Lo scopo dell'esercitazione è quello di progettare e addestrare una rete neurale del tipo MLP, applicata alla risoluzione del problema della regressione non lineare. La regressione è una tecnica fondamentale nel machine learning che ha come obiettivo quello di stimare una variabile continua(target) a partire da un insieme di variabili indipendenti(features). Il focus dell'analisi è l'individuazione della topologia ottimale di rete che sia in grado di minimizzare l'errore di generalizzazione, evitando i fenomeni di overfitting e underfitting.

1. Librerie Utilizzate

Per l'implementazione ho dovuto utilizzare le seguenti librerie Python:

- **NUMPY:** è la libreria per il calcolo scientifico. In particolare fornisce supporto per array multidimensionali e matrici di grandi dimensioni. Nell'esercitazione viene utilizzata per gestire quest'ultima casistica (dataset in forma matriciale).

- **MATPLOTLIB:** è la libreria utilizzata per la generazione dei grafici, in particolare per effettuare il confronto tra Test Error e Training Error, facilitando l'analisi visiva delle performance.

- **SCIKIT-LEARN:** è la libreria per l'implementazione dell'algoritmo MLPRegressor, fornisce gli strumenti per la standardizzazione degli input e la validazione.

- **WARNINGS:** gestisce messaggi di avviso senza fermare il programma. L'obiettivo era quello di pulire l'output da avvisi che non compromettono l'esecuzione del codice ma che distraggono visivamente dai risultati.

Ho scelto di non utilizzare la libreria Neurolab, preferendo invece MLPRegressor della libreria scikit-learn. La decisione è dovuta dalla mancanza di ottimizzazioni da parte di Neurolab, che avrebbero comportato un lavoro più laborioso: implementare manualmente loop di training, il calcolo del MSE e la gestione della convergenza dei grafici. Al contrario tutte queste operazioni vengono gestite in modo nativo e efficiente da scikit-learn.

A sostegno di questa decisione, ho notato la scarsa compatibilità di Neurolab con le versioni più recenti di Python e di Numpy. In particolare, durante la compilazione del codice si verificava un errore riferito alla seguente istruzione che nelle versioni precedenti si poteva utilizzare:

```
out_minmax = [-np.Inf, np.Inf]
```

Nelle versioni ≥ 2.0, **`np.Inf`** non esiste più e bisogna usare **`np.inf.`** Questo la linea di codice compare nel file **trans.py** che è uno dei file dei quali è costituita la libreria Neurolab. Il problema è stato facilmente risolvibile andando ad installare la versione precedente di Numpy compatibile con Neurolab. In alternativa avrei potuto modificare manualmente il file 'trans.py'.

2. Lettura Dataset

La versione finale della fase di caricamento del dataset è stata progettata andando a valutare la robustezza e la gestione controllata di eventuali anomalie nel file. Il primo tentativo di lettura è stato effettuato con la funzione '**`loadtxt`**', che però richiede che tutte le righe abbiano lo stesso numero di colonne, condizione non sempre rispettata. Con '**`np.genfromtxt`**', invece, si possono leggere file CSV che potrebbero contenere: righe vuote, valori mancanti e colonne deformate. L'argomento '**`invalid_raise=False`**' impedisce al parser di interrompere l'esecuzione in presenza di righe malformate. Questo è utile quando si lavora con dataset reali o semi-strutturati e permette al codice di completare il caricamento inserendo NaN nelle posizioni problematiche.

Dopo la lettura, tutte le righe che contengono almeno un valore non numerico o mancante (NaN) vengono rimosse. Con la seguente linea di codice si genera una maschera booleana che seleziona esclusivamente le righe completamente valide.

```
data = np.genfromtxt(DATASET_PATH, delimiter=",", invalid_raise=False)
data = data[~np.isnan(data).any(axis=1)]
```

- Garantire che l'addestramento avvenga su un dataset numericamente coerente.
- Evitare problemi di normalizzazione e rimozione del rumore informativo

Il dataset è strutturato in modo che:

- le prime colonne rappresentino le **feature** di input del modello,
- l'ultima colonna rappresenti il **target** da apprendere

3. Suddivisione DataSet

```
4. X_train, X_test, y_train, y_test = train_test_split(
       X, y, test_size = 0.3, random_state=RANDOM_STATE
   )
```

Ho utilizzato la funzione train_test_split della libreria scikit-learn. La suddivisione è stata configurata secondo le linee guida dell'esercitazione: 70% training(usato per l'ottimizzazione dei pesi tramite backpropagation) e 30% test(usato per la valutazione finale). Prima della divisione, l'algoritmo esegue una permutazione casuale delle righe

5. Normalizzazione

La funzione di attivazione logistic(sigmoide) è particolarmente sensibile alla scala dei dati: per valori di input molto grandi tende a saturarsi, causando il fenomeno del Vanishing Gradient (gradiente che si annulla), che rallenta o blocca l'apprendimento. L'operazione di normalizzazione serve per limitare questo fenomeno ed è stata effettuata con '`StandardScaler`', che trasforma le feature della matrice X in modo che abbiano media pari a 0 e deviazione standard pari a 1. Questa scelta migliora la stabilità numerica e favorisce una convergenza più rapida del modello, evitando che la sigmoide entri in saturazione.

Per quanto riguarda il **target y**, non è stata applicata alcuna scalatura. Questa scelta è corretta perché, nel caso della regressione, l'uscita della rete utilizza una funzione di attivazione lineare, che permette di prevedere valori su qualsiasi scala. Lasciare **y** nella sua unità originale consente di calcolare l'errore quadratico medio (MSE) nella stessa unità dei dati, rendendo il risultato facilmente interpretabile. Sarei stato obbligato a normalizzare **y** se avessi imposto una funzione di attivazione come la sigmoide anche per l'ultimo neurone. Senza la correzione sarebbe impossibile raggiungere la soluzione.

6. Training Model

Ho utilizzato MLPRegressor, un modello di rete neurale feed-forward fornito da scikit-learn. L'algoritmo implementa una rete neurale utilizzando backpropagation e supporta funzionalità avanzate come la validazione interna e l'early stopping, che consente di interrompere l'addestramento quando l'errore smette di migliorare. Ho dichiarato alcuni valori di configurazione all'inizio del codice per agevolare eventuali modifiche dovute ai test e per renderlo più leggibile. Di seguito la spiegazione di alcune linee di codice caratteristiche di questo modello:

- `random_state = RANDOM_STATE` → imposta il seme del generatore di numeri casuali utilizzato dal modello. La rete neurale inizializza i pesi in modo casuale e se non controlliamo questo comportamento, ogni esecuzione produrrebbe risultati diversi. Il seme garantisce la confrontabilità tra modelli con architetture differenti. Il numero 7 non ha particolare significato, avrei potuto utilizzare qualsiasi costante.
- `activation = 'logistic'` → è la funzione di attivazione sigmoide che genera output compresi tra 0 e 1. Introduce non linearità ed è continua e derivabile, caratteristiche fondamentali per la backpropagation.
- `max_iter = MAX_ITERATIONS` → numero di iterazioni scelto in base a test.
- `learning_rate_init = LEARNING_RATE` → valore scelto con i dati normalizzati e permette una convergenza abbastanza veloce senza risultare instabile. Particolare attenzione a questo parametro perché se lo scelgo troppo basso rischio di cadere in tempi di esecuzione molto

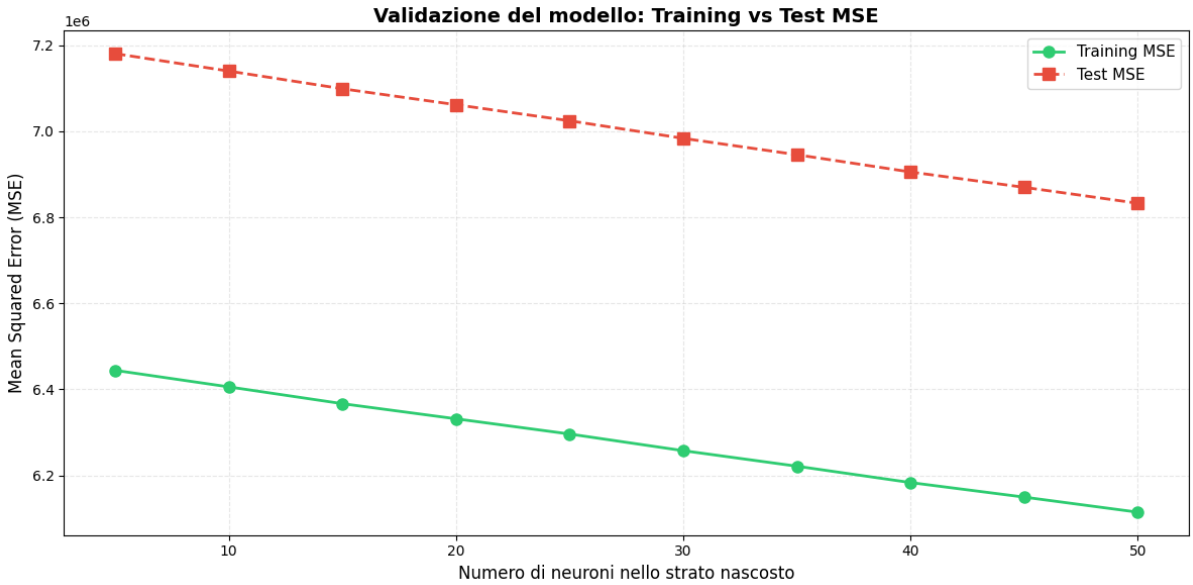
elevati, ottenendo però maggiore precisione. Se lo scelgo troppo alto, rischio di saltare il punto ottimale avendo una convergenza più rapida, ma meno precisa.

- `tol = TOLERANCE` → impedisce che la rete continui ad addestrarsi anche quando la funzione di errore migliora di quantità irrilevanti.
- `early_stopping=False` → nessuna validazione interna, quindi, alleno solo sul 70% senza riservare una quota per la validazione

7. Considerazioni e Test

7.1. UNDERFITTING

Il grafico evidenzia un chiaro stato di **mancata convergenza** dovuto a un arresto prematuro dell'addestramento. L'algoritmo di ottimizzazione (Discesa del Gradiente) è stato interrotto a 50 iterazioni, un numero computazionale insufficiente per permettere ai pesi della rete di assestarsi verso il minimo della funzione di costo.



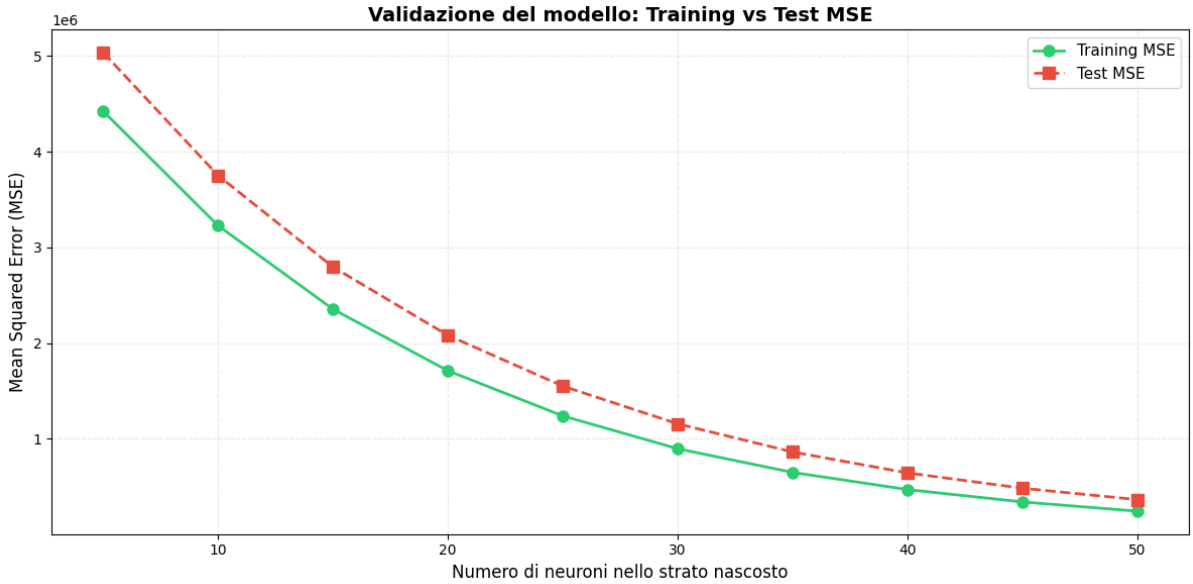

```
LEARNING_RATE = 0.0001
TOLERANCE = 1e-8
MAX_ITERATIONS = 50
```

Modello Ottimale	
Neuroni	50
MSE Test	68.328.283.111
MSE Train	61.145.780.453
Gap (overfitting)	7.182.502.658

Neuroni	MSE Train	MSE Test
5	6444358.17.00	7180686.27.00
10	6405956.53.00	7140218.28.00
15	6366960.32.00	7099062.13.00
20	6331734.17.00	7062031.24.00
25	6296291.49.00	7024705.24.00
30	6257625.14.00	6983866.52.00
35	6221525.33.00	6945801.29.00
40	6183516.05.00	6905601.32.00
45	6149625.07.00	6869923.43.00
50	6114585.33.00	6832879.51.00

7.2. OVERFITTING

In questa seconda casistica, il modello mostra un comportamento opposto al caso precedente. Grazie a un numero adeguato di iterazioni, la rete ha raggiunto la convergenza. Osserviamo che all'aumentare della complessità il **Mean Squared Error** sul Training set tende asintoticamente a zero. Questo indica che il modello ha acquisito una capacità rappresentativa tale da 'memorizzare' perfettamente i dati di addestramento (Overfitting). L'errore di test rimane basso, e l'aderenza quasi perfetta al training set ci suggerisce che il modello sta iniziando a modellare anche il rumore statistico presente nei dati, perdendo potenzialmente capacità di generalizzazione su dati futuri più rumorosi. Un errore di training nullo è molto sospetto poiché il rumore non è del tutto eliminabile (bayesiano). Se avessi utilizzato un valore di tolleranza del tipo 1e-4 avrei ottenuto un risultato migliore in quanto la rete si sarebbe fermata prima di imparare anche il rumore.



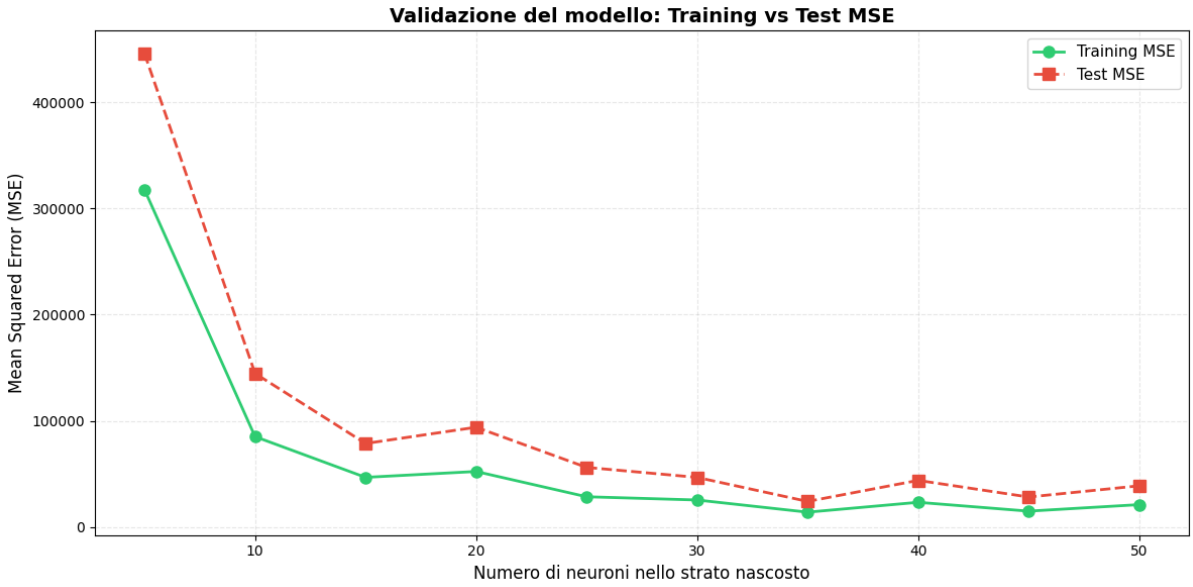

```
LEARNING_RATE = 0.01
TOLERANCE = 1e-8
MAX_ITERATIONS = 3000
```

Modello ottimale	
Neuroni	50
MSE Test	3.650.337.477
MSE Train	2.443.852.001
Gap (Overfitting)	1.206.485.476

Neuroni	MSE Train	MSE Test
5	44.212.010.082	50.337.066.942
10	32.257.995.324	37.494.568.039
15	23.537.659.967	27.944.006.398
20	17.111.674.164	20.805.371.371
25	12.394.493.187	15.507.733.690
30	8.967.703.244	11.565.595.073
35	6.493.448.712	8.628.608.577
40	4.701.503.848	6.441.694.529
45	3.410.578.718	4.844.462.734
50	2.443.852.001	3.650.337.477

7.3. INSTABILE

Questa configurazione è **inutilizzabile in produzione**. Un modello affidabile deve mostrare stabilità: piccoli cambiamenti negli iperparametri dovrebbero corrispondere a piccoli cambiamenti nelle prestazioni. La causa di questo fenomeno è un **Learning Rate** η eccessivo. Durante la *Backpropagation* la rete aggiorna i pesi cercando di scendere verso il minimo della funzione di errore. Con un valore così alto, l'algoritmo compie "balzi" troppo ampi sulla superficie di errore. Invece di scivolare gradualmente verso il fondovalle (il minimo globale). Il codice identifica automaticamente il modello con **35 neuroni** come "Ottimale", ma rappresenta solamente un caso fortuito.



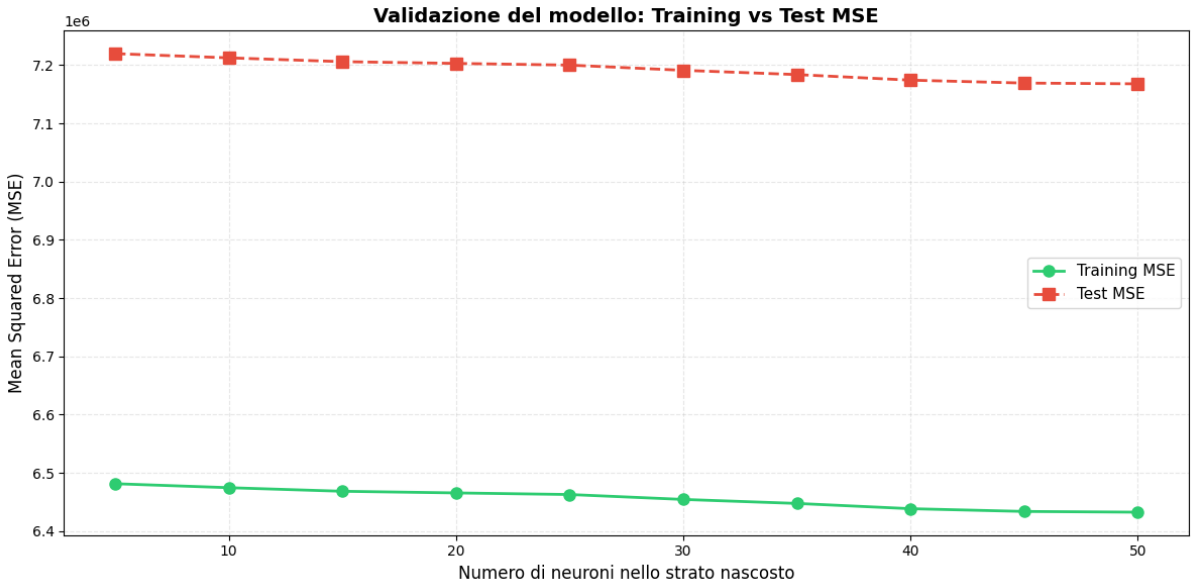
```
LEARNING_RATE = 0.8
TOLERANCE = 1e-8
MAX_ITERATIONS = 500
```

Modello ottimale	
Neuroni	35
MSE Test	240.705.593
MSE Train	139.631.818
Gap (Overfitting)	101.073.774

Neuroni	MSE Train	MSE Test
5	3.176.455.770	4.458.681.374
10	851.116.727	1.446.709.801
15	467.640.321	786.697.241
20	522.908.358	939.607.323
25	284.827.274	561.597.237
30	254.315.493	467.375.500
35	139.631.818	240.705.593
40	232.552.115	439.097.668
45	149.337.885	282.419.993
50	210.678.533	388.008.334

7.4. CONVERGENZA LENTA

La causa di questi andamenti che migliorano in maniera “infinitesimale” è il valore del tasso di apprendimento insufficiente. Anche se il gradiente è elevato (siamo lontani dal minimo), il moltiplicatore η è talmente piccolo che il "passo" di aggiornamento risulta microscopico. In 1000 iterazioni, la rete ha compiuto mille "micro-passi", percorrendo una distanza insignificante sulla superficie di errore. Se avessi usato un valore come 1e-4 per la tolleranza la rete avrebbe pensato ancora prima di essere arrivata e avrei ottenuto un risultato più inefficiente dal punto di vista computazionale.



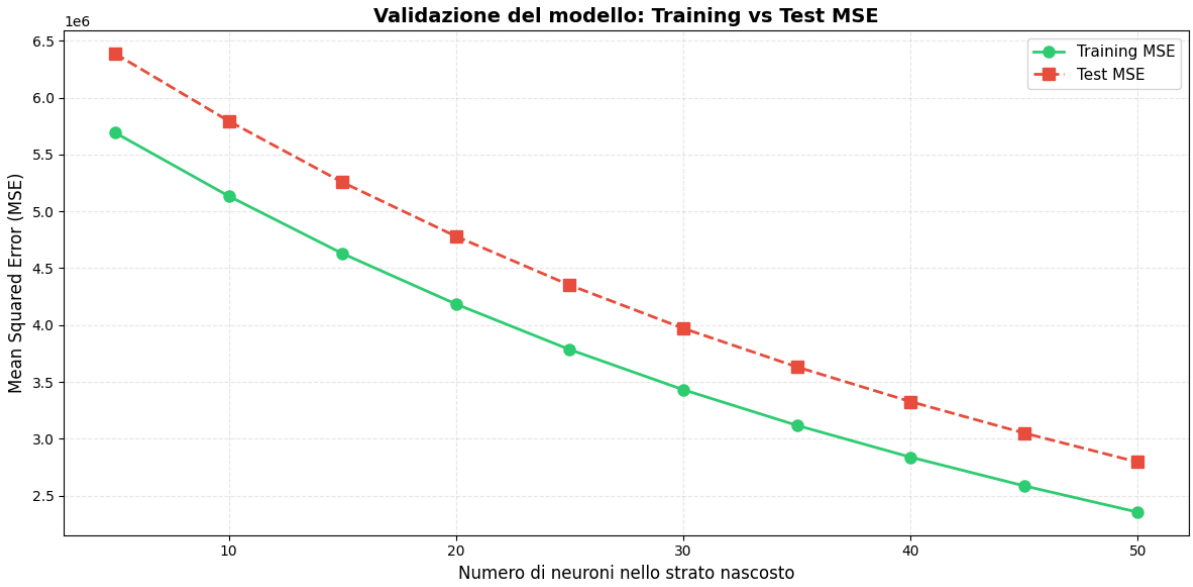

```
LEARNING_RATE = 0.0001
TOLERANCE = 1e-8
MAX_ITERATIONS = 1000
```

Modello ottimale	
Neuroni	50
MSE Test	71.678.310.995
MSE Train	64.325.664.409
Gap (overfitting)	7.352.646.585

Neuroni	MSE Train	MSE Test
5	64.814.221.422	72.196.619.055
10	64.746.013.113	72.124.459.218
15	64.684.494.029	72.059.145.648
20	64.656.153.799	72.029.079.551
25	64.628.714.273	71.999.844.890
30	64.544.434.924	71.910.409.969
35	64.475.681.374	71.837.239.854
40	64.385.508.658	71.741.594.330
45	64.338.182.176	71.691.481.639
50	64.325.664.409	71.678.310.995

7.5. OTTIMALE

Poiché la curva non ha ancora raggiunto un plateau (asintoto orizzontale), ma continua a scendere, è probabile che il modello possa beneficiare ulteriormente di un aumento della complessità (es. 60-70 neuroni) o di un numero maggiore di iterazioni. Al momento non ha ancora saturato completamente la sua capacità di apprendimento sui dati disponibili. La pendenza "delicata" delle curve indica che il tasso di apprendimento è calibrato secondo giusti parametri.




```
LEARNING_RATE = 0.01
TOLERANCE = 1e-4
MAX_ITERATIONS = 1000
```

Modello ottimale	
Neuroni	50
MSE Test	27.943.715.832
MSE Train	23.540.144.670
Gap (overfitting)	4.403.571.162

Neuroni	MSE Train	MSE Test
5	56.914.402.067	63.864.516.125
10	51.340.409.782	57.939.745.297
15	46.289.240.122	52.558.837.328
20	41.842.131.629	47.801.212.146
25	37.843.988.166	43.511.047.547
30	34.313.167.405	39.710.303.399
35	31.177.117.360	36.317.254.374
40	28.379.034.385	33.264.805.426
45	25.858.353.106	30.499.352.821
50	23.540.144.670	27.943.715.832